

# A programação por IA vai substituir todos os programadores? Vejamos a situação real do setor

Zhihu · 2026-09-19

Desde o fim de 2025 e o início de 2026, quando o efeito da programação por IA melhorou enormemente, boa parte do trabalho de codificação dos programadores — quando não quase todo ele — passou a ser substituída pela IA. O fenômeno gerou pânico e desespero entre os programadores. Ao mesmo tempo, as empresas de IA, aproveitando os avanços nesse campo, exageraram sem limites as capacidades da IA, apregoando que em um ou dois anos todos os colarinhos-brancos estarão desempregados. Os diversos veículos de mídia própria seguiram a onda, difundindo o pânico e fabricando ansiedade.

Sobre essa questão, é preciso examinar as coisas com seriedade. A programação por IA pode substituir todos os programadores? Se não pode, qual é a taxa real de substituição?

Na verdade, numa resposta que escrevi alguns meses atrás (A primeira espada da IA corta os programadores: eles vão ficar em massa sem emprego?), já dei meu veredito: a IA não pode substituir todo o trabalho humano, e a taxa de substituição de ponta a ponta não passa de 30%.

Quando sistemas de máquinas inteligentes centrados na IA entrarem em larga escala na produção, o trabalho de codificação manual praticamente desaparecerá. É claro que o desenvolvimento de código central, de arquitetura complexa ou de exigências de desempenho muito altas, talvez ainda exija forte intervenção humana; mas a grande massa do desenvolvimento comum de código será integralmente substituída pela IA. Quantas pessoas, concretamente, serão substituídas? Pelo meu uso real e pelo retorno de outras pessoas no Zhihu, a eficiência sobe algo entre 20% e 50%; se a programação por IA se implantar normalmente (hoje ainda em grande experimentação no setor), estimando-se em 60% a proporção de desenvolvedores puros, seriam cortados cerca de 12% a 30% dos postos de programador.

A implantação em larga escala da programação por IA já dura ao menos meio ano, sobretudo na China e nos EUA, onde praticamente todas as grandes empresas exigem que os funcionários usem agentes de IA no desenvolvimento. Então, qual é o efeito real?

Segundo o relatório da McKinsey<sup>[1]</sup>, a produtividade pode subir cerca de 16% a 30%, em linhas gerais em consonância com meu juízo anterior.

McKinsey analyzed nearly 300 publicly traded companies to understand how AI is reshaping software development.<sup>[2]</sup> We found that a small group of top performers—roughly the top quintile—are achieving 16–30 percent improvements in productivity, time to market, and customer experience, along with 31–45 percent gains in software quality.

知乎 @最最遥远的路

Uma pesquisa recente<sup>[2]</sup>

As ferramentas de IA generativa para desenvolvimento de software avançaram vertiginosamente: em poucos anos passaram do simples preenchimento automático de código a agentes de codificação plenamente autônomos (coding agents). Esses ganhos de capacidade se convertem em aumento de produtividade no nível das tarefas no mundo real? E, mesmo que se convertam, o ganho de eficiência no nível da tarefa implica aumento do produto final? Essas questões são cruciais para entender o impacto econômico da IA. Por meio de um estudo de eventos emparelhados com desenvolvedores, descobrimos que, com a iteração das ferramentas de codificação por IA, seu efeito de produtividade no nível da tarefa se torna cada vez mais acentuado: o preenchimento automático aumentou o volume semanal de commits em cerca de 30%; os agentes síncronos (sync agents) elevaram esse efeito acumulado a cerca de 180%; e os agentes assíncronos (async agents), a cerca de 240%. No entanto, à medida que se sobe na hierarquia até a etapa de produção, esses ganhos se atenuam bruscamente: embora os agentes síncronos tenham aumentado as linhas de código em 958% e os pull requests em 86%, o número de versões efetivamente publicadas cresceu apenas 20%. A atividade geral de codificação no GitHub mostra a mesma regularidade.

## AI Boosts Coders' Output, But Not Their Releases

| TOOL TYPE          | LINES OF CODE | COMMITTS | RELEASES |
|--------------------|---------------|----------|----------|
| Autocomplete Tools | 228%          | 36%      | 10%      |
| Sync Agents        | 741%          | 109%     | 20%      |
| Async Agents       | 658%          | 34%      | n/a      |

Percentages reflect how much a developer's output rose after adopting that category of AI tool, compared with their own activity before adoption. AI delivers its biggest boost at the code-writing stage, while human-driven steps — reviewing, merging, releasing — absorb most of that advantage before software actually ships.

Source: Adapted from M. Demirel, L. Musolff, and L. Yang, "Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools," 2026.

知乎 @最靠谱的路  
MIT MANAGEMENT  
SLIGAN SCHOOL

Fonte: <https://mitsloan.mit.edu/ideas-made-to-matter/ai-boosts-worker-productivity-does-translate-to-final-outputs>

Não são só os estudos que confirmam os limites da programação por IA: os desenvolvedores e gestores que usam agentes de IA na prática têm essa sensação direta. A atitude dos programadores diante da programação por IA já saiu do choque inicial e, com a aplicação real na produção, eles vão reconhecendo os limites da IA. Por exemplo, num post de blog<sup>[3]</sup> de Cal Newport (professor de ciência da computação da Universidade de Georgetown), um desenvolvedor que, após programar com Claude Code, viu a produtividade subir várias vezes e exclamou «really, really good», depois de sofrer dois incidentes em produção causados pelo código da IA e quase perder o emprego, desiludiu-se com a programação por IA e voltou a escrever à mão o código central, usando a IA apenas para testes, scripts e coisas do tipo. Esse desenvolvedor considera que a programação à moda antiga é lenta, mas confiável, com qualidade garantida. O autor observa que essa situação não é isolada: muitos engenheiros do setor expressam opiniões semelhantes.

Onde está o problema da programação por IA? Um programador, em seu blog<sup>[4]</sup>, cita o artigo «Decadência das restrições: a fragilidade dos agentes LLM na geração de código de backend»<sup>[5]</sup>:

Nossa pesquisa revela um fenômeno de «decadência das restrições»: à medida que aumentam as exigências estruturais, o desempenho dos agentes cai de forma acentuada. De tarefas de referência a tarefas com requisitos plenamente especificados, as configurações de melhor desempenho caíram, em média, 30 pontos percentuais na taxa de aprovação de asserções, e algumas configurações mais fracas chegaram perto de zero. A análise de sensibilidade em relação aos frameworks revelou diferenças de desempenho significativas: os agentes vão bem em frameworks minimalistas e explícitos (como Flask), mas, em ambientes que dependem fortemente de convenções (como FastAPI e Django), o desempenho médio cai muito. Por fim, a análise de erros mostra que defeitos na camada de dados (por exemplo, construção errada de consultas e exceções de tempo de execução de ORM) são a principal origem desses problemas. Este estudo sublinha que satisfazer simultaneamente requisitos funcionais e estruturais continua sendo um desafio crucial e ainda não resolvido para os agentes de programação.

O CTO de uma empresa indiana também mencionou, num artigo recente<sup>[6]</sup>:

O código da IA é escrito rápido, mas, pelos problemas acima, os desenvolvedores precisam examinar com cuidado o código gerado pela IA antes de integrá-lo ao repositório. Daí decorrem dois problemas. Primeiro, a velocidade de revisão do código é bem menor que a de geração, o que desconta boa parte do ganho de eficiência da IA. Segundo, os novos programadores não têm oportunidade de escrever código com as próprias mãos e não desenvolvem suas capacidades; quando a velha geração de programadores for demitida ou se aposentar, a revisão de código ficará ainda mais lenta e poderá até deixar de detectar problemas.

Pior ainda: algumas empresas depositam grandes esperanças na programação por IA e comprimem drasticamente os prazos de desenvolvimento, de modo que os programadores não têm tempo algum para examinar o código com seriedade. Para o desenvolvedor, a empresa também não é dele; se o chefe exige rapidez, ele revisa por cima ou nem revisa e já faz o merge. Se der problema depois de entrar em produção, no pior caso pede demissão. Seja pela fadiga de revisão, seja pela revisão insuficiente, o resultado final é que o código que chega ao ambiente de produção esconde uma pilha de bugs e de code smells, e quem é responsável por manter esse software, na prática, não conhece o código. Toda essa dívida técnica produz o chamado «abismo da manutenibilidade» (The Maintainability Gap)<sup>[7]</sup>.

A programação assistida por IA continua a avançar com força, e os desenvolvedores mais entusiasmados atingem volumes de código sem precedentes. No entanto, uma análise de quatro anos de dados de mudanças de código mostra que os indicadores de manutenibilidade do código estão em queda: as chamadas de função entre arquivos (que expressam reutilização de código) caíram 35%; em comparação com o nível de 2022, a migração de linhas de código em refatorações caiu 70%, e o trabalho de manutenção sobre código legado de longa data caiu 74%. Ao mesmo tempo, observamos uma tendência preocupante de alta em vários fenômenos: o comportamento de «copiar/colar» dentro de um mesmo commit aumentou 41%, a duplicação de blocos de código cresceu 81%, as estruturas de codificação que mascaram erros aumentaram 47% e a taxa de churn de código em duas semanas subiu 15%. Embora a produção de código de fato tenha aumentado muito, a dívida técnica que se acumula com ela não pode ser ignorada; essa dívida concentra-se sobretudo em desenvolvedores que ainda não perceberam que certos padrões de codificação põem em risco a manutenibilidade de longo prazo do repositório.

Sobre a dívida técnica, o artigo «Debt Behind the AI Boom»<sup>[8]</sup> mostra que «os code smells são o tipo de problema mais comum, com até 89,3%»; «mais de 15% dos commits gerados pelos diversos assistentes de programação por IA introduzem ao menos um problema. Mais importante: entre os problemas introduzidos pela IA, 22,7% ainda estão presentes na versão mais recente do repositório». «Esses achados indicam que o código gerado por IA pode trazer custos de manutenção de longo prazo aos projetos reais de software e evidenciam a necessidade de reforçar a garantia de qualidade no desenvolvimento assistido por IA.»

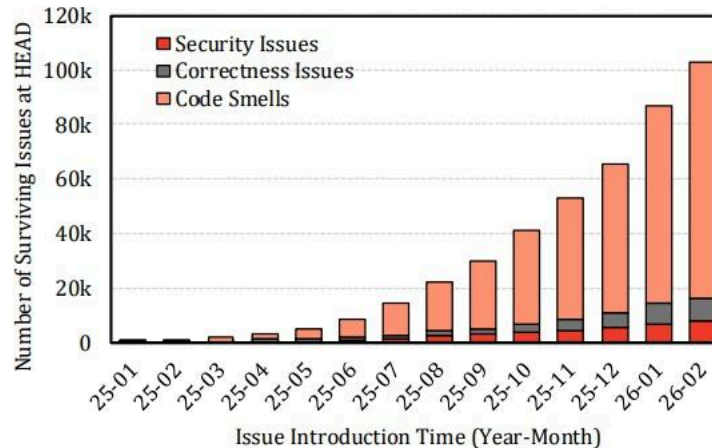


Fig. 9: Cumulative growth of AI-introduced issues over time, by issue type. 知乎 @最最遥远的路

De: «Debt Behind the AI Boom»

E o artigo «Faster Code, Deeper Debt?»<sup>[9]</sup> aponta que os LLM tendem a agravar as formas tradicionais de dívida técnica (sobretudo dívida de código, de projeto e de documentação), além de introduzir novas modalidades de dívida típicas dos LLM. «Notavelmente, identificamos uma "dívida de integração rápida": sacrificar a qualidade em nome da velocidade, de modo que o código gerado depressa provoca reações em cadeia, gerando dívida de governança e elevando o custo de manutenção de longo prazo. Outras categorias emergentes incluem dívida de prompts, dívida ética, dívida de dados e dívida de rastreabilidade, que refletem os desafios próprios das aplicações de LLM.»

A programação por IA não só acumula dívida técnica, como também introduz outras dívidas. A cientista da computação Margaret-Anne Storey, no artigo «From Technical Debt to Cognitive and Intent Debt»<sup>[10]</sup>, aponta:

Lendo até aqui, creio que o leitor já tem sua resposta. Em muitas respostas minhas (como: Você acha que a programação por IA realmente substituirá os programadores no futuro? e Como avaliar a previsão do responsável de IA da Microsoft de que «nos próximos 18 meses a maioria dos empregos de colarinho branco será completamente substituída pela IA»?) repito sempre um ponto de vista: a IA mudará profundamente a forma de trabalhar dos colarinhos-brancos, mas é impossível que elimine todos os programadores, e mais impossível ainda que ponha todos os colarinhos-brancos no olho da rua. A realidade comprovou que meu ponto de vista está correto.

## Referências

- The AI revolution in software development <https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>
- ^Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools.  
[https://download.ssrn.com/document/6843118.pdf?response-content-disposition=inline&X-Amz-Security-](https://download.ssrn.com/document/6843118.pdf?response-content-disposition=inline&X-Amz-Security-Token=IQoJb3JpZ2luX2VjEir%2F%2F%2F%2F%2F%2F%2F%2FwEaCXvZWVhc3QtMSJHMEUCIA4KlxaTLIbYG8l%2Amz-Algorithm=AWS4-HMAC-SHA256&X-Amz-Date=20260919T025315Z&X-Amz-SignedHeaders=host&X-Amz-Expires=300&X-Amz-Credential=ASIAUPUUPRWEYZTNLYVN%2F20260919%2Fus-east-1%2Fs3%2FAws4_request&X-Amz-Signature=4ba6ead523ab5e286d4d945af8b9fbf80ada4d1e8fd200c179fedea48do732b3&abstractId=6843118)
- On AI Coding and Its Discontents <https://calnewport.com/on-ai-coding-and-its-discontents/>
- I Spent 6 Months Reviewing AI Code. Here's the Ugly Truth [https://medium.com/@haseeb\\_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33eob25](https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33eob25)
- Constraint Decay: The Fragility of LLM Agents in Backend Code Generation  
<https://arxiv.org/pdf/2605.06445>
- The Productivity Illusion at the Heart of Enterprise AI Coding <https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding/>
- The Maintainability Gap: AI Code Quality in 2026  
[https://www.gitclear.com/the\\_ai\\_code\\_quality\\_maintainability\\_gap](https://www.gitclear.com/the_ai_code_quality_maintainability_gap)
- Debt Behind the AI Boom: A Large-Scale Empirical Study of AI-Generated Code in the Wild  
<https://arxiv.org/abs/2603.28592>
- Faster Code, Deeper Debt? A Multivocal Literature Review on Technical Debt and Its Early Signs in LLM-Assisted Software Development <https://arxiv.org/abs/2606.14796>
- From Technical Debt to Cognitive and Intent Debt: Rethinking Software Health in the Age of AI  
<https://arxiv.org/abs/2603.22106>

[1] The AI revolution in software development: a revolução da IA no desenvolvimento de software.

<https://www.mckinsey.com/capabilities/tech-and-ai/our-insights/the-ai-revolution-in-software-development>

中文原文: Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools.

[3] On AI Coding and Its Discontents: sobre a programação por IA e seus descontentamentos.

<https://calnewport.com/on-ai-coding-and-its-discontents/>

[https://medium.com/@haseeb\\_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25](https://medium.com/@haseeb_sohail/i-spent-6-months-reviewing-ai-code-heres-the-ugly-truth-b81fe33e0b25)

<https://arxiv.org/pdf/2605.06445>

<https://cacm.acm.org/blogcacm/the-productivity-illusion-at-the-heart-of-enterprise-ai-coding/>

[https://www.gitclear.com/the\\_ai\\_code\\_quality\\_maintainability\\_gap](https://www.gitclear.com/the_ai_code_quality_maintainability_gap)

<https://arxiv.org/abs/2603.28592>

<https://arxiv.org/abs/2606.14796>

<https://arxiv.org/abs/2603.22106>

本文原发表于知乎专栏/回答（中文）：<https://zhuanlan.zhihu.com/p/2084561490930045240>；作者：最最遥远的路：<https://www.zhihu.com/people/zui-zui-yao-yuan-de-lu-32/posts>